

# APEX Planet Internship — Task 3 Report

Web Application Security Assessment & Lab Demonstration

**Intern:** Maaz Abbasi

**Track:** Cybersecurity & PenTesting

**Window:** Days 25–36

## 1. Laboratory Topology & Setup

Task 3 builds upon an isolated, local multi-node virtual environment simulating real-world web application vulnerabilities. Components interact within a secure host-only subnet.

| Component   | Role                    | Software / Details                                           |
|-------------|-------------------------|--------------------------------------------------------------|
| Kali Linux  | Attacker / Testing Node | Firefox, Burp Suite Community, cURL, Nmap                    |
| DVWA Target | Vulnerable Web App      | Apache, PHP, MySQL/MariaDB (Damn Vulnerable Web Application) |

## 2. SQL Injection (SQLi) Assessment

**Framework:** Vulnerability → Demonstration → Impact → Mitigation

### Vulnerability Overview & Demonstration

At low security level, the application accepts user input via the `?id=` parameter without sanitization or escaping. Normal requests like `?id=1` fetch single records. Testing boolean conditions such as `1' OR '1'='1` alters the logical structure of the SQL statement:

```
SELECT first_name, last_name FROM users WHERE id = 1' OR '1'='1;
```

This evaluates to true for all database rows, resulting in complete table enumeration in the HTTP response.

### Impact & Mitigation

**Impact:** Full database compromise, unauthorized data extraction, and potential authentication bypass.

**Prevention:** Separate data from query execution using PDO parameterized statements:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?"); $stmt->execute([$user_input]);
```

### 3. Cross-Site Scripting (XSS) Evaluation

**Scope:** Reflected XSS, Stored XSS, and Multi-Layer Defensive Controls.

#### Reflected XSS Demonstration

Tested via DVWA → XSS (Reflected) using payload `<script>alert('XSS')</script>`. The server immediately echoes unencoded input inside the HTTP response, causing the browser to execute arbitrary JavaScript in the context of the user session.

#### Stored XSS Demonstration

Tested via DVWA → XSS (Stored) by submitting malicious script payloads into guestbooks or comment fields. The payload persists inside the database and executes automatically whenever any victim visitor loads the affected page.

| XSS Type  | Execution Vector                    | Persistence                      |
|-----------|-------------------------------------|----------------------------------|
| Reflected | URL Parameter / Immediate Echo      | Transient (Single Request)       |
| Stored    | Database Storage / Victim Rendering | Persistent (Multi-User Exposure) |

### 4. XSS Defensive Architecture & Mitigations

Effective defense requires layered security controls rather than single-point fixes:

- **Input Validation:** Strict allow-listing of expected data types and formats at the application perimeter.
- **Output Encoding:** Neutralizing special characters before rendering markup:

```
htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8');
```

- **Content Security Policy (CSP):** Restricting resource loading origins to mitigate script execution risks:

```
Content-Security-Policy: default-src 'self';
```

## 5. Cross-Site Request Forgery (CSRF)

CSRF forces an authenticated victim browser to execute unwanted state-changing actions (e.g., password modification) without user consent.

```
/password-change.php?password_new=test123&password_conf=test123
```

### CSRF Token Protection & Verification

Secure forms implement unguessable, session-bound anti-CSRF tokens. The application validates requests by comparing submitted tokens against session state:

| Request State              | Token Status | Server Action            |
|----------------------------|--------------|--------------------------|
| Request without Token      | Missing      | 403 Forbidden / Rejected |
| Request with Invalid Token | Mismatch     | 403 Forbidden / Rejected |
| Request with Valid Token   | Matched      | Accepted & Processed     |

## 6. File Inclusion Vulnerabilities (LFI & RFI)

### Local File Inclusion (LFI)

Improper path validation allows directory traversal sequences (../../../../etc/passwd) to expose sensitive operating system files or application configuration parameters.

### Remote File Inclusion (RFI)

Occurs when input parameters load external resources. Within our isolated lab network, including http://LAB-SERVER/test.txt verified remote inclusion capabilities, presenting severe Remote Code Execution (RCE) risks if server-side script evaluation is permitted.

## 7. Burp Suite Interception & Automated Intruder

---

Configured Firefox proxy through Burp Suite Community Edition to intercept, inspect, and modify HTTP traffic during DVWA authentication. **Burp Intruder** was utilized against isolated target parameters with custom wordlists (e.g., admin, password, test) to measure response codes, content lengths, and behavioral anomalies.

## 8. HTTP Security Headers Hardening

---

Audited target responses and configured Apache headers module (`sudo a2enmod headers`) to enforce robust security posture:

```
Header always set X-Content-Type-Options "nosniff" Header always set Referrer-Policy "strict-origin-when-cross-origin" Header always set Content-Security-Policy "default-src 'self'"
```

## 9. Evidence Checklist & GitHub Repository Structure

---

All lab artifacts, raw request logs, and verification screenshots are organized in the GitHub repository structure:

```
Task-3-Web-Application-Security/ ├── README.md ├── SQL-Injection/ (Normal, Injection, Mitigation) ├── XSS/ (Reflected, Stored, CSP) ├── CSRF/ (Token Analysis) ├── File-Inclusion/ (LFI & RFI captures) ├── Burp-Suite/ (Proxy & Intruder logs) ├── Security-Headers/ (curl -I & Apache config) └── reports/Task-3-Web-Security-Report.pdf
```